

H i g h C o h e s i o n

High Cohesion Principle

Keep related work together

DEFINITION

Assign responsibilities so each class keeps a focused, strongly related set.

Cohesion measures how single-minded a class is — how strongly its methods and fields pull toward one purpose. High cohesion means everything in the class belongs together and serves the same goal; low cohesion is a grab-bag where unrelated duties share a class only by accident of history. It is the constant partner of Low Coupling.

WHY IT MATTERS

Highly cohesive classes are easy to name, understand, test, and reuse, and they change for fewer reasons. Low cohesion spreads one concept across many classes and pulls unrelated dependencies into each, which raises coupling. Most "this class is hard to name" or "I touch it for everything" pain is really a cohesion problem.

— How it works

Cohesion	How closely a class's responsibilities relate — ideally all serving one clear purpose.
Goal	Focused, single-purpose classes you can name in one phrase and reason about in isolation.
Means	Extract unrelated duties into their own classes; group together only what truly belongs.

HOW TO APPLY

1. Try to describe the class's job in one sentence with no "and".
2. If you need an "and", or methods touch unrelated fields, you have multiple responsibilities.
3. Extract each cluster of related methods plus its data into its own focused class.
4. Balance against coupling — do not split until classes turn anemic and chatty.

LITMUS TEST

Can you describe the class's job in one sentence without using "and"? Do most methods use most of its fields? Several "and"s, or fields touched by only one method, signal low cohesion.

— Smell → fix

Order computes tax, renders a PDF, and sends email — three unrelated concerns in one class:

```
class Order {  
  total() { /* domain */ }  
  renderPdf() { /* presentation */ } // unrelated  
  sendEmail() { /* messaging */ } // unrelated  
}
```

↓ SPLIT BY CONCERN

```
class Order { total() { /* domain only */ } }  
class PayslipPdf { render(o: Order) {} } // presentation  
class Mailer { send(o: Order) {} } // messaging  
// each class now has one focused responsibility
```

Each class now has one reason to exist. They still compose when needed, but a change to PDF layout no longer risks the tax logic.

USE WHEN

- ✓ A class doing many unrelated things
- ✓ Balanced against Low Coupling

AVOID WHEN

- ▲ Splitting so far you get scattered, anemic classes

— Trade-offs & pitfalls

PROS

- + Readable, testable, maintainable
- + Easier reuse

CONS

- Over-splitting raises coupling and coordination

COMMON PITFALLS

- ▲ Over-splitting into one-method classes — cohesion collapses into coordination overhead and higher coupling.
- ▲ "Manager", "Helper", and "Util" buckets that accrete unrelated functions over time.
- ▲ Mixing layers in one class (domain plus persistence plus presentation) — three reasons to change.

WHERE YOU SEE IT

- Order (domain) · OrderRepository (persistence) · InvoicePdf (rendering) instead of one Order doing all three.
- A React component split into view plus a custom hook for data logic.
- Small value objects (Money, Email) whose every method serves the one concept.

SEE ALSO

SRP	the SOLID phrasing of cohesion — "one reason to change"
Low Coupling	its constant counter-force — balance, do not maximise either
Information Expert	placing behaviour with its data usually raises cohesion
SoC	Separation of Concerns is cohesion at architectural scale

— Interview & sources

High Cohesion vs SRP?

Nearly the same lens: SRP says "one reason to change", High Cohesion asks "how related are the duties". SRP is the SOLID framing of cohesion.

How do you measure cohesion?

Informally: name the class without "and", check whether methods share its fields. Formal metrics like LCOM exist, but the one-sentence test catches most cases.

Can cohesion be too high?

Splitting until every class has one trivial method gives anemic, scattered design and raises coupling. Cohesion is balanced against coupling, not pushed to an extreme.

What's a quick smell of low cohesion?

A class you can't name without "and", a "Manager / Util" catch-all, or methods that each use a different subset of fields. These signal duties that belong in separate classes.

REMEMBER

Each class should hold one job's worth of strongly related work.

SOURCES

Craig Larman — "Applying UML and Patterns" (3rd ed., 2004): High Cohesion

Stevens, Myers & Constantine — "Structured Design" (1974): the cohesion scale